BUDGETBRIDGE

Milestone 03 Testing Summary Report

CP476B - Internet Computing | Group 6

Application	BudgetBridge Personal Expense Tracker
Test date	July 22, 2026
Target platform	Node.js 18+ / MySQL 8.x / modern browser
Automated result	12 passed, 0 failed
Final due date	July 24, 2026

Purpose

This report documents the final test strategy, automated results, manual verification plan, known limitations, and execution instructions for the BudgetBridge full-stack application.

1. Test Strategy and Environment

BudgetBridge was tested at three levels: input validation and security utilities, authenticated API integration, and manual end-to-end browser/database verification. The automated suite uses the same controller and store interfaces as the MySQL application, but runs with an isolated in-memory store so tests are repeatable and do not modify a developer database.

Test objectives

- Verify secure registration, login, logout, password hashing, and session invalidation.
- Verify transaction and category CRUD behavior, user ownership isolation, filtering, reports, and settings.
- Verify invalid inputs produce clear 4xx responses instead of unsafe or inconsistent database writes.
- Verify code structure, JavaScript syntax, HTML structure, and clean-machine setup instructions.
- Identify honest limitations before the final demonstration.

Executed environment

Component	Configuration / Evidence
Node.js	v22.16.0; compatible with the documented Node.js 18+ requirement.
Automated runner	Node.js built-in node:test runner through npm test.
API/database abstraction	Production MySQL store plus isolated memory store used for repeatable automated API tests.
Static checks	All backend, front-end JavaScript, and test files passed node --check. All HTML pages were parsed and checked for required document elements and duplicate IDs.
Database target	MySQL 8.x schema with InnoDB tables, primary keys, foreign keys, checks, unique rules, and indexes.

Command used

```
npm test
```

Result: 12 tests passed, 0 failed, 0 skipped. Total automated run time was approximately 0.59 seconds in the test environment.

2. Automated Test Results

ID	Test case	Result
AT-01	Register, create transaction, update, report, delete, logout	PASS
AT-02	Block access to another user's transaction	PASS
AT-03	Custom category create, update, delete; protect defaults	PASS
AT-04	Filter by type/date and search by description	PASS
AT-05	Update profile and password; invalidate old session	PASS
AT-06	Reject duplicate account and negative transaction amount	PASS
AT-07	Password hash verifies correct password only	PASS
AT-08	Session tokens are random and stored as hashes	PASS
AT-09	Normalize email and accept strong registration input	PASS
AT-10	Reject weak password	PASS
AT-11	Reject zero/negative transaction amount	PASS
AT-12	Reject invalid report/filter date range	PASS

Security and data-integrity checks covered

- Passwords are hashed with scrypt and compared using a timing-safe comparison.
- Session tokens are random; only SHA-256 token hashes are stored by the data layer.
- Protected API routes reject unauthenticated requests.
- User IDs are included in transaction/category queries, preventing cross-account access.
- Amounts, dates, category IDs, names, email addresses, search strings, and passwords are validated server-side.
- MySQL queries use placeholders rather than concatenating user input.

3. Manual End-to-End Verification Plan

The following checklist must be executed in MySQL mode on the computer used for the final recording. This is the final clean-machine verification sequence and doubles as the demo rehearsal.

ID	Manual action	Expected result
MT-01	Run schema.sql in MySQL 8.x	All five tables are created with no SQL errors.
MT-02	npm install and npm start	Server starts and reports MySQL mode.
MT-03	Register a new user	User, default categories, and session row are created.
MT-04	Log out and log back in	Correct password works; incorrect password is rejected.
MT-05	Add income and expense records	Records persist after refresh and appear only for the user.
MT-06	Edit and delete a transaction	Database-backed update/delete reflected in dashboard totals.
MT-07	Search and filter transactions	Results match type, category, date range, and search text.
MT-08	Create/edit/delete a custom category	Custom category CRUD works; protected/default rules display errors.
MT-09	Generate report for a date range	Totals and category aggregation match stored transactions.
MT-10	Update profile and password	Profile persists; password change logs out existing session.

MT-11	Responsive browser check	Core pages remain usable at desktop and mobile widths.
MT-12	Run npm test	All 12 automated tests pass before recording.

Defect-handling rule

If any manual case fails, do not record the final demo. Record the defect in the activity blog/Kanban board, fix it, rerun the affected test and the complete automated suite, and then update the test result before submission.

4. Known Issues, Limitations, and Final Assessment

Area	Status / limitation
Deployment	The project is designed for local course demonstration and is not deployed to a public production host.
Currency	Amounts are displayed in Canadian dollars only.
Authentication scope	Email password reset and multi-factor authentication are outside the approved project scope.
Reporting	Reports provide totals and category aggregation, not forecasting or bank reconciliation.
Browser storage	Financial records are no longer stored in localStorage; the final app uses the API/database. The browser stores only the HTTP-only session cookie.
Final environment check	The group must complete the MySQL manual checklist on the recording computer and verify the shared demo link before submission.

Final assessment

The executed automated evidence demonstrates that BudgetBridge business logic, validation, authentication, CRUD workflow, reporting, account isolation, and password/session behavior are functioning as designed. The code also passed JavaScript syntax and HTML structure checks. The remaining pre-submission responsibility is a clean-machine MySQL smoke test and the required recorded demo, which must use the production MySQL mode rather than the in-memory test mode.

How to reproduce the automated results

- Open a terminal in the project root.
- Run npm install.
- Run npm test.
- Confirm the summary shows 12 passed and 0 failed.

Approval checklist

☐ Testing report reviewed by the QA lead.

☐ MySQL manual verification completed.

☐ Known limitations stated honestly in the demo.

☐ Video link tested from a signed-out/private browser window.